

Architecture Documentation: Tasks by Me Application

1. Introduction

"Tasks by Me" is an open-source web application designed to help users easily view their assigned Microsoft Planner tasks. It integrates directly with a Microsoft 365 tenant using the Microsoft Graph API, providing a streamlined experience for users to sign in with their Microsoft 365 accounts and see a consolidated view of their tasks. The application prioritizes speed by maintaining a per-user, volatile local cache of task data. Developed with Node.js 22, Express, and plain web technologies, "Tasks by Me" is highly portable and can be deployed across various environments.

2. Key Architectural Components

The "Tasks by Me" application is structured around several interconnected components:

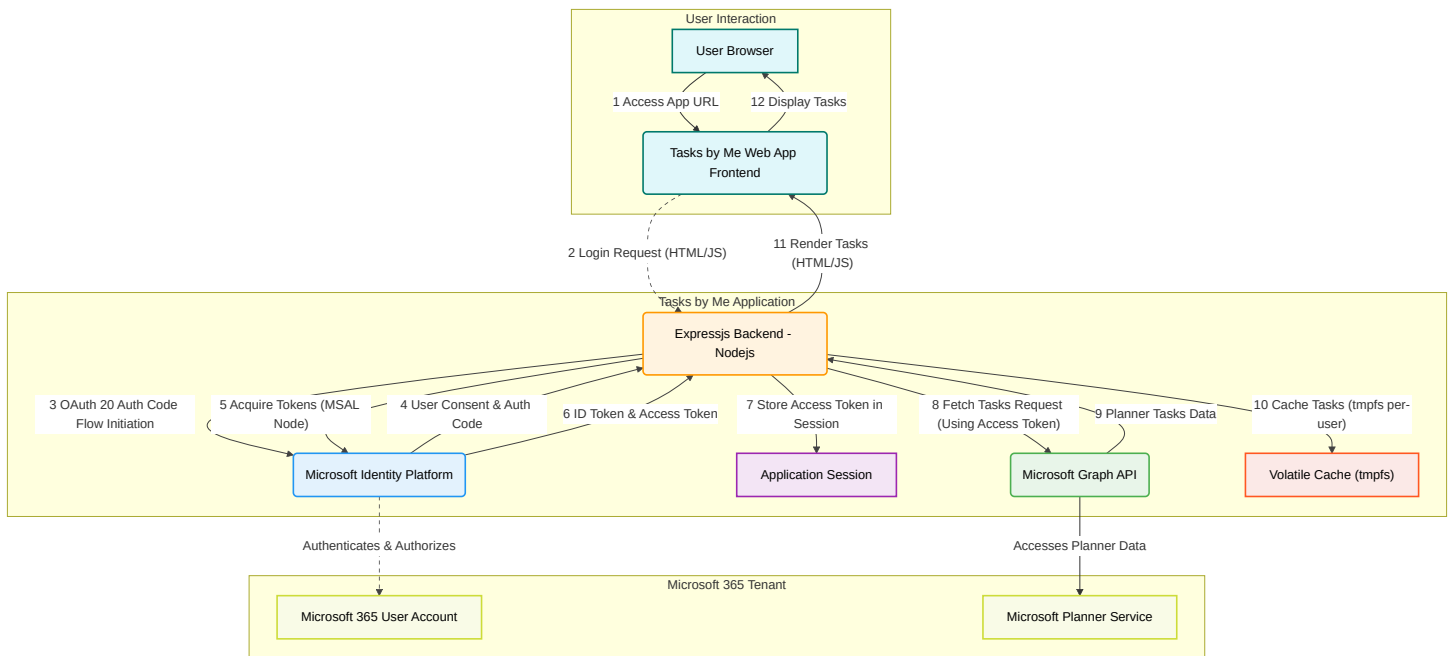
- **User Browser (Frontend):**
 - **Technology:** Plain HTML, CSS, and JavaScript.
 - **Functionality:** Handles initial user interaction, displays the login prompt, and renders the fetched Planner tasks. It communicates with the Express.js backend for all application logic and data retrieval.
- **Express.js Backend (Node.js 22):**
 - **Technology:** Node.js 22 with the Express.js framework.
 - **Functionality:** This is the core application logic layer. It manages user authentication via MSAL Node , interacts with the Microsoft Graph API, implements the caching mechanism, and serves the frontend assets.
- **Microsoft Identity Platform (External Service):**
 - **Role:** Handles user authentication and authorization for Microsoft 365 accounts.
 - **Interaction:** Facilitates the OAuth 2.0 Authorization Code Flow, issues ID tokens and access tokens, and manages user consent for application permissions.
- **Microsoft Graph API (External Service):**
 - **Role:** The unified API endpoint for accessing Microsoft 365 data.
 - **Interaction:** "Tasks by Me" queries this API specifically to retrieve Planner tasks and related metadata for the authenticated user.
- **Volatile Cache (tmpfs):**
 - **Technology:** In-memory file system (tmpfs).
 - **Functionality:** Stores a per-user cache of Planner tasks and associated metadata to improve application responsiveness and reduce repeated calls to the Microsoft Graph API. Being

tmpfs , this cache is volatile and is cleared upon application restart or server reboot.

- **Microsoft 365 Tenant (External Environment):**

- **Role:** The organizational environment providing user accounts and Planner data.

Diagram



3. Data Flow and Application Logic

The application follows a clear flow from user interaction to task display:

1. **Initial Access:** A user navigates to the "Tasks by Me" application URL in their browser. The frontend presents a login button.
2. **Login Initiation:** The user clicks the login button. The frontend initiates a request to the Express.js backend to start the authentication process.
3. **OAuth 2.0 Authorization Code Flow:**
 - The Express.js backend, using the `@azure/msal-node` library, redirects the user's browser to the Microsoft Identity Platform for authentication.
 - The user goes through their organization's standard login process (e.g., entering credentials, MFA if configured).
 - If it's their first time logging in, the user is presented with the requested application permissions (`User.Read` , `User.ReadBasic.All` , `Tasks.Read` , `Tasks.Read.Shared`) and prompted for consent.
 - Upon successful authentication and consent, the Microsoft Identity Platform redirects the user's browser back to the Express.js backend with an authorization code.

4. **Token Acquisition:** The Express.js backend uses the received authorization code to request an ID token and an access token from the Microsoft Identity Platform.
5. **Session Management:** The acquired access token is securely stored in the user's session on the Express.js backend. Token expiration and renewal are handled automatically by `MSAL Node`.
6. **Task Retrieval:** With a valid access token, the Express.js backend makes authenticated calls to the Microsoft Graph API (specifically, endpoints related to Planner tasks) to fetch all tasks owned by or shared with the logged-in user.
7. **Caching:** The retrieved Planner task data is stored in the per-user volatile cache residing on a `tmpfs` file system. This improves subsequent retrieval speed.
8. **Task Display:** The Express.js backend renders the fetched (and potentially cached) task data and sends it back to the user's browser. The frontend then displays the tasks to the user.
9. **Error Handling:** If no tasks are found, an appropriate message is displayed rather than an error. Graph API rate limits or throttling are currently handled by showing a generic error message to the user, advising them to retry later.

4. Security Considerations

- **OAuth 2.0 Authorization Code Flow:** This is a highly recommended and secure flow for web applications, preventing direct exposure of tokens to the client-side.
- **MSAL Node Library:** The use of `MSAL Node` (`@azure/msal-node`) ensures adherence to Microsoft's recommended practices for secure authentication and token management.
- **Session-Based Token Storage:** Storing access tokens in server-side sessions prevents them from being accessible on the client-side, reducing the risk of client-side token theft.
- **Required Permissions:** The application requests specific, minimal permissions (`User.Read` , `User.ReadBasic.All` , `Tasks.Read` , `Tasks.Read.Shared`) necessary for its functionality, adhering to the principle of least privilege. Users are explicitly asked for consent.
- **Configuration:** Sensitive configurations, such as application client IDs and secrets, are managed via environment variables (`.env` for development/local environments), promoting secure handling and preventing hardcoding.

5. Deployment Model

"Tasks by Me" is designed for flexible deployment and can be hosted in various environments:

- **Virtual Machines (VMs):** Can be deployed directly on a VM, running Node.js.
- **Containers:** A Docker image is available (GitHub Container Registry), enabling deployment in containerized environments.
- **Orchestrators:** Compatible with container orchestration platforms like Kubernetes.

- **Platform as a Service (PaaS):** Can be easily deployed to PaaS offerings such as Azure App Service, which simplifies hosting and scaling.

The application relies on environment variables for configuration, making it adaptable to the specific configuration mechanisms of each deployment platform.

6. Scalability and Performance

- **Local Caching:** The per-user `tmpfs` cache significantly enhances performance by reducing the need for repeated Microsoft Graph API calls for frequently accessed task data. This makes the application feel more responsive to the user.
- **Volatile Nature:** The `tmpfs` cache is volatile, meaning data is lost on restarts. For very high-scale scenarios where cache persistence or sharing across multiple instances is critical, an external, persistent caching solution (e.g., Redis) might be considered as a future enhancement.
- **Rate Limiting:** Currently, the application provides a basic error message for Microsoft Graph API rate limits. For enterprise-level deployments with heavy usage, implementing retry logic with exponential backoff and potentially a queueing mechanism would be a beneficial enhancement to gracefully handle API throttling.

7. Open Source and Licensing

"Tasks by Me" is an open-source project hosted on GitHub at <https://github.io/maxoffice/tasksbyme>. It is licensed under the **MIT License**, promoting free use, modification, and distribution. This open-source nature allows for community contributions, audits, and transparency.